

Tugas 11: Algoritma K-Mean Clustering

Aisya Az Zahra - 0110224053

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: azzahrasya1705@gmail.com

Abstract. Praktikum ini membahas penerapan algoritma DBSCAN pada dataset Airbnb New York City untuk mengelompokkan listing berdasarkan fitur numerik seperti harga, minimum nights, jumlah review, jumlah listing host, dan ketersediaan. Proses mencakup instalasi library, pembersihan data, visualisasi spasial dengan heatmap dan choropleth, uji normalitas, normalisasi menggunakan RobustScaler, penentuan parameter eps melalui K-Distance Graph, serta evaluasi clustering dengan silhouette score 0.1049 yang menunjukkan pemisahan cluster lemah. Hasil menunjukkan 132 cluster dengan mayoritas data pada kategori "Kurang" (47.319 listing), sedikit "Baik" (370 listing), dan noise (1.190 listing), memberikan insight tentang pola listing terjangkau di Manhattan dan Brooklyn.

```
[4] # 1. INSTALLASI LIBRARY
    !pip install folium contextily geopandas plotly scikit-learn

Requirement already satisfied: folium in /usr/local/lib/python3.12/dist-packages (0.20.0)
Requirement already satisfied: contextily in /usr/local/lib/python3.12/dist-packages (1.7.0)
Requirement already satisfied: geopandas in /usr/local/lib/python3.12/dist-packages (1.1.1)
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (5.24.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: branca<0.6.0 in /usr/local/lib/python3.12/dist-packages (from folium) (0.8.2)
Requirement already satisfied: Jinja2>=2.9 in /usr/local/lib/python3.12/dist-packages (from folium) (3.1.6)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from folium) (2.0.2)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from folium) (2.32.4)
Requirement already satisfied: xyzservices in /usr/local/lib/python3.12/dist-packages (from folium) (2025.11.0)
Requirement already satisfied: geopy in /usr/local/lib/python3.12/dist-packages (from contextily) (2.4.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from contextily) (3.10.0)
Requirement already satisfied: mercantile in /usr/local/lib/python3.12/dist-packages (from contextily) (1.2.1)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from contextily) (11.3.0)
Requirement already satisfied: rasterio in /usr/local/lib/python3.12/dist-packages (from contextily) (1.4.3)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from contextily) (1.5.2)
Requirement already satisfied: pyogrio>=0.7.2 in /usr/local/lib/python3.12/dist-packages (from geopandas) (0.11.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from geopandas) (25.0)
Requirement already satisfied: pandas>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from geopandas) (2.2.2)
Requirement already satisfied: pyproj>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from geopandas) (3.7.2)
Requirement already satisfied: shapely>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from geopandas) (2.1.2)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from plotly) (9.1.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2>=2.9->folium) (3.0.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=2.0.0->geopandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=2.0.0->geopandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=2.0.0->geopandas) (2025.2)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.4->contextily) (2025.11.12)
Requirement already satisfied: geographiclib>=1.52 in /usr/local/lib/python3.12/dist-packages (from geopy>=2.4.1->contextily) (2.1)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0->contextily) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0->contextily) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0->contextily) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0->contextily) (1.4.9)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.10.0->contextily) (3.2.5)
Requirement already satisfied: click>=3.0 in /usr/local/lib/python3.12/dist-packages (from rasterio>=1.4.3->contextily) (8.3.1)
Requirement already satisfied: affine in /usr/local/lib/python3.12/dist-packages (from rasterio>=1.4.3->contextily) (2.4.0)
Requirement already satisfied: attrs in /usr/local/lib/python3.12/dist-packages (from rasterio>=1.4.3->contextily) (25.4.0)
Requirement already satisfied: clig>=0.5 in /usr/local/lib/python3.12/dist-packages (from rasterio>=1.4.3->contextily) (0.7.2)
Requirement already satisfied: click-plugins in /usr/local/lib/python3.12/dist-packages (from rasterio>=1.4.3->contextily) (1.1.1.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.4->folium) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.4->folium) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.4->folium) (2.5.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=2.0.0->geopandas) (1.17.0)
```

Penjelasan :

Kode :

Kode diatas berfungsi buat menginstall semua library yang bakal dipakai selama praktikum. Library ini dipakai untuk mengolah data, membuat visualisasi peta, dan

menjalankan algoritma DBSCAN. Perintah pip install digunakan untuk memastikan semua paket sudah tersedia di lingkungan Google Colab sebelum proses analisis dimulai.

Output :

Output menunjukkan kalau semua library yang diinstall ternyata sudah ada di sistem Colab. Itu sebabnya muncul tulisan requirement already satisfied. Artinya paket-paket itu sudah siap dipakai tanpa perlu diinstall ulang. Tidak ada error, jadi proses instalasi dinyatakan berhasil.

1.

```
[5]
✓ 2s
# menghubungkan colab dengan google drive
from google.colab import drive
drive.mount('/content/gdrive')

Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).
```

Penjelasan :

Kode :

Kode diatas digunakan untuk menghubungkan google colab dengan google drive.

Output :

Output menunjukan bahwa kode inputan telah berhasil dan telah menghubungkan google colab dengan google drive.

2.

```
[6]
✓ 18s
# 2. IMPORT LIBRARY LENGKAP
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import RobustScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.neighbors import NearestNeighbors
from scipy.stats import shapiro
import folium
from folium.plugins import HeatMap
import geopandas as gpd
import contextily as ctx
import plotly.express as px
from mpl_toolkits.axes_grid1 import make_axes_locatable
sns.set_style('whitegrid')
```

Penjelasan :

Kode :

Kode diatas berfungsi untuk mengimpor semua library yang bakal dipakai selama proses analisis. Setiap library punya fungsi masing-masing. Pandas dan numpy dipakai untuk mengolah data. Matplotlib dan seaborn dipakai untuk membuat visualisasi grafik. DBSCAN dan library sklearn lainnya dipakai untuk proses clustering, scaling, PCA, dan evaluasi model. Scipy dipakai untuk uji normalitas. Folium dan HeatMap dipakai untuk membuat peta interaktif. Geopandas dan contextily dipakai untuk visualisasi peta berbasis koordinat. Plotly dipakai untuk visualisasi interaktif, sedangkan make_axes_locatable dipakai untuk pengaturan layout warna pada grafik tertentu. Pengaturan sns.set_style digunakan untuk membuat tampilan grafik lebih rapi dan konsisten.

3.

```

sns.set_style('whitegrid')

[7]
✓ 2s
# Loading Dataset
df = pd.read_csv("/content/gdrive/MyDrive/Praktikum02/Praktikum11/data/AB_NYC_2019.csv")

```

Penjelasan :

Kode :

Kode diatas digunakan untuk membaca file dataset Airbnb yang disimpan di Google Drive. Fungsi read_csv dari pandas dipakai untuk membuka file dengan format CSV dan mengubahnya menjadi sebuah DataFrame. DataFrame ini yang nantinya akan dipakai untuk melakukan analisis, visualisasi, dan proses clustering. Jalur file yang ditulis harus sesuai lokasi penyimpanan dataset di Drive supaya file bisa berhasil dibaca.

4.

```

[8]
✓ 0s
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48895 entries, 0 to 48894
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0   id                                    48895 non-null  int64
1   name                                48879 non-null  object
2   host_id                             48895 non-null  int64
3   host_name                           48874 non-null  object
4   neighbourhood_group                 48895 non-null  object
5   neighbourhood                       48895 non-null  object
6   latitude                           48895 non-null  float64
7   longitude                           48895 non-null  float64
8   room_type                           48895 non-null  object
9   price                               48895 non-null  int64
10  minimum_nights                      48895 non-null  int64
11  number_of_reviews                   48895 non-null  int64
12  last_review                         38843 non-null  object
13  reviews_per_month                  38843 non-null  float64
14  calculated_host_listings_count      48895 non-null  int64
15  availability_365                    48895 non-null  int64
dtypes: float64(3), int64(7), object(6)
memory usage: 6.0+ MB

```

Penjelasan :

Kode :

Kode diatas digunakan untuk mengetahui informasi yang ada pada dataset yang telah dimasukan pada kode sebelumnya.

Output :

Output yang muncul menunjukkan kalau dataset berisi 48.895 baris dan 16 kolom. Terdapat beberapa kolom yang bertipe angka seperti price, minimum_nights, dan latitude, serta beberapa kolom teks seperti name, neighbourhood_group, dan room_type. Dari informasi ini juga terlihat kalau beberapa kolom tidak terisi penuh, contohnya name dan host_name yang ada beberapa nilai kosong, serta last_review dan reviews_per_month yang punya cukup banyak missing value. Informasi ini membantu menentukan langkah pembersihan data selanjutnya.

The screenshot shows a Jupyter Notebook cell with the code `df.head()` executed. The output is a table with 15 columns: `id`, `name`, `host_id`, `host_name`, `neighbourhood_group`, `neighbourhood`, `latitude`, `longitude`, `room_type`, `price`, `minimum_nights`, `number_of_reviews`, `last_review`, `reviews_per_month`, `calculated_host_listings_count`, and `availability_365`. The first 5 rows of data are displayed.

	id	name	host_id	host_name	neighbourhood_group	neighbourhood	latitude	longitude	room_type	price	minimum_nights	number_of_reviews	last_review	reviews_per_month	calculated_host_listings_count	availability_365
0	2539	Clean & quiet apt home by the park	2787	John	Brooklyn	Kensington	40.64749	-73.97237	Private room	149	1	9	2016-10-19	0.21	6	365
1	2595	Skyli Midtown Castle	2945	Jennifer	Manhattan	Midtown	40.75362	-73.98377	Entire home/apt	225	1	45	2019-05-21	0.38	2	355
2	3647	THE VILLAGE OF HARLEM - NEW YORK!	4632	Elisabeth	Manhattan	Harlem	40.80802	-73.94190	Private room	150	3	0	NaN	NaN	1	365
3	3831	Cozy Entire Floor of Brownstone	4869	LisaRocanne	Brooklyn	Clinton Hill	40.68514	-73.95976	Entire home/apt	89	1	270	2019-07-05	4.64	1	194
4	5022	Entire Apt- Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Harlem	40.79851	-73.94399	Entire home/apt	80	10	9	2018-11-19	0.10	1	0

5.

Penjelasan :

Kode :

Kode diatas digunakan untuk melihat 5 baris awal yang ada pada dataset.

Output :

Output yang muncul menampilkan lima baris pertama dari dataset Airbnb. Setiap baris mewakili satu listing atau tempat menginap. Dari hasil tersebut terlihat informasi seperti nama tempat, lokasi borough dan neighbourhood, koordinat latitude dan longitude, jenis kamar, harga per malam, jumlah review, sampai ketersediaan dalam setahun. Di baris kedua terlihat beberapa nilai kosong pada kolom `last_review` dan `reviews_per_month`, yang menunjukkan bahwa tidak semua listing memiliki data lengkap. Informasi ini bisa dijadikan dasar untuk proses cleaning berikutnya.

The screenshot shows a Jupyter Notebook cell with the following code:

```
# 4. DATA CLEANING (missing + duplikat)
print("\n=== DATA CLEANING ===")
print(f"Missing values: {df.isnull().sum().sum()}")
print(f"Duplikat: {df.duplicated().sum()}")
df_clean = df[['id', 'name', 'host_id', 'neighbourhood_group', 'latitude', 'longitude',
               'room_type', 'price', 'minimum_nights', 'number_of_reviews',
               'calculated_host_listings_count', 'availability_365']].dropna()
df_clean = df_clean.drop_duplicates()
print(f"Shape setelah cleaning: {df_clean.shape}")
```

The output of the code is:

```
=== DATA CLEANING ===
Missing values: 20141
Duplikat: 0
Shape setelah cleaning: (48879, 12)
```

6.

Penjelasan :

Kode :

Kode diatas digunakan buat ngecek kondisi data sebelum dipakai ke tahap analisis berikutnya. Pertama, kode menghitung jumlah nilai yang hilang di seluruh kolom dan jumlah baris yang duplikat. Setelah itu, dibentuk sebuah DataFrame baru bernama `df_clean` yang hanya mengambil kolom yang dianggap penting untuk analisis. Pada tahap ini, baris yang mengandung nilai kosong langsung dibuang dengan `dropna`, lalu baris yang duplikat juga dihapus menggunakan `drop_duplicates`. Langkah ini dilakukan supaya data yang dipakai bersih dan tidak mengganggu proses visualisasi atau pemodelan DBSCAN.

Output :

Output menunjukkan bahwa total nilai yang hilang di dataset ada cukup banyak, yaitu 20141 nilai kosong. Namun jumlah baris yang duplikat adalah nol, artinya tidak ada duplikasi dalam data asli. Setelah dilakukan pembersihan dan penyaringan kolom, jumlah baris yang tersisa adalah 48879 dengan 12 kolom. Ini berarti data yang hilang hanya berasal dari kolom yang tidak digunakan pada analisis, sehingga jumlah baris tidak banyak berkurang meskipun dilakukan `dropna`.

```
[11]
✓ Os # Data Cleaning (Pembersihan Data)
df.isnull().sum()

id      0
name    16
host_id  0
host_name 21
neighbourhood_group 0
neighbourhood 0
latitude 0
longitude 0
room_type 0
price    0
minimum_nights 0
number_of_reviews 0
last_review 10052
reviews_per_month 10052
calculated_host_listings_count 0
availability_365 0

dtype: int64
```

7.

Penjelasan :

Kode :

Perintah `df.isnull().sum()` dipakai untuk menghitung berapa banyak nilai kosong di setiap kolom dalam dataset. Dengan perintah ini, kita bisa melihat kolom mana yang punya missing value dan perlu diproses lebih lanjut. Bagian ini penting sebelum analisis karena data yang belum bersih bisa mempengaruhi hasil visualisasi maupun proses clustering.

Output :

Output menunjukkan jumlah nilai kosong di masing-masing kolom. Terlihat ada beberapa kolom yang punya missing value cukup banyak, misalnya `last_review` dan `reviews_per_month` yang masing-masing punya lebih dari sepuluh ribu nilai kosong. Kolom `name` dan `host_name` juga punya sedikit nilai kosong. Sementara kolom lainnya tidak memiliki missing value sama sekali. Informasi ini membantu menentukan langkah cleaning seperti mengisi nilai kosong, menghapus kolom tertentu, atau mengurangi kolom yang tidak diperlukan.

```
[12]
✓ Os df.duplicated().sum()

np.int64(0)
```

8.

Penjelasan :

Kode :

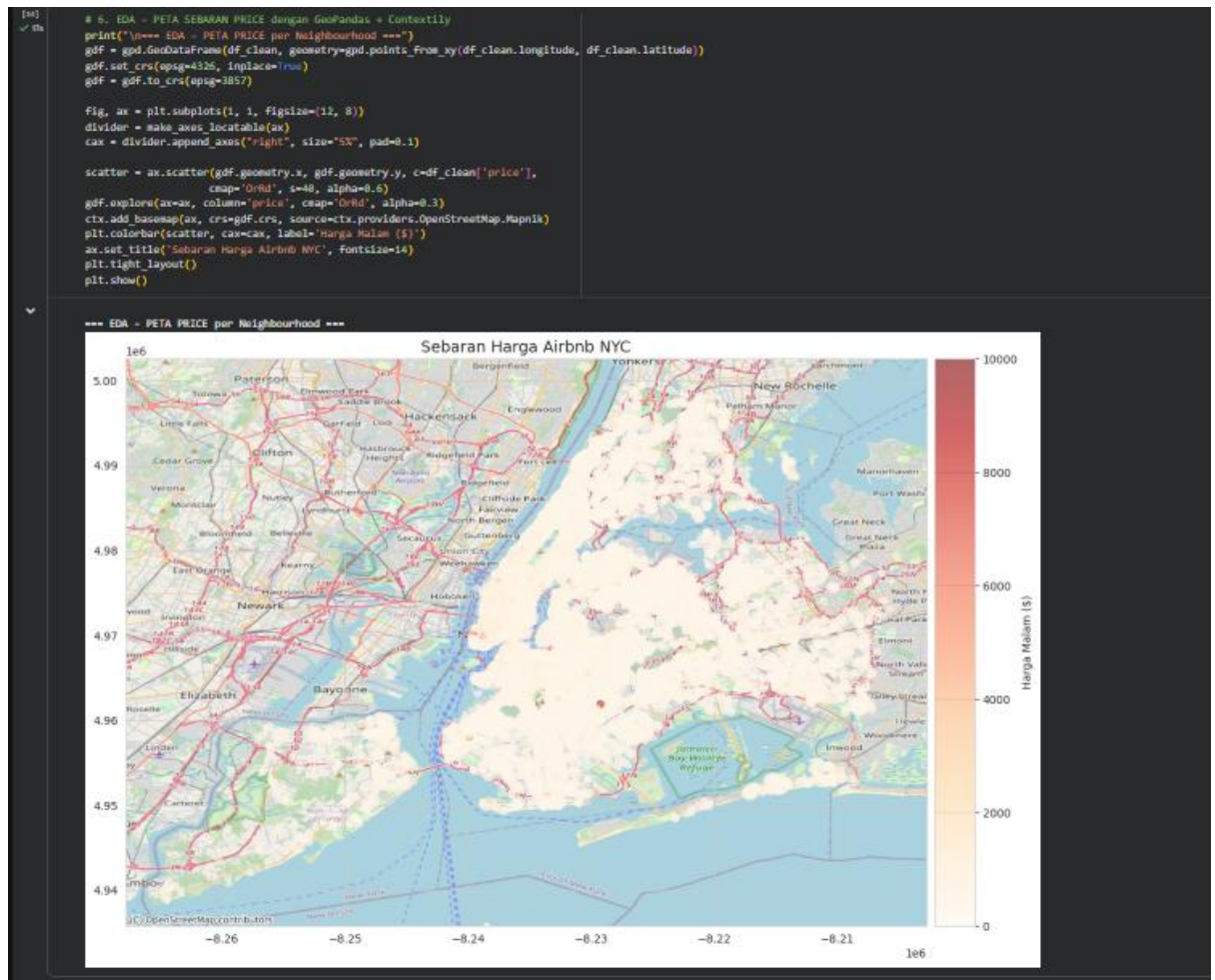
Nilainya 0, artinya di dataset tidak ditemukan baris yang sama persis. Dataset bersih dari duplikasi.



- folium.Map() membuat peta dasar dengan posisi tengah di NYC.
- HeatMap() membuat layer heatmap berdasarkan koordinat latitude-longitude.
- radius dan blur mengatur sebaran panas agar terlihat lebih halus.
- m1.save() menyimpan hasil dalam bentuk HTML.
- m1 menampilkan peta langsung di kolab.

- merah/kuning = jumlah listing sangat padat.
- hijau = jumlah listing sedang.
- biru = jumlah listing sedikit.

Heatmap menunjukkan bahwa Airbnb paling terkonsentrasi di area Manhattan dan Brooklyn, yang terlihat sebagai zona merah-kuning pada peta.



10.

Penjelasan :

Kode :

Pada bagian ini dilakukan visualisasi awal berupa peta persebaran harga Airbnb di New York City. Visualisasi ini bertujuan untuk melihat pola distribusi harga dalam bentuk peta sehingga area dengan harga tinggi dan rendah bisa terlihat secara jelas. Data latitude dan longitude diubah menjadi titik koordinat, kemudian ditampilkan di atas peta dasar menggunakan OpenStreetMap.

- Membuat GeoDataFrame agar titik latitude-longitude bisa digunakan sebagai objek geometri.
- Mengatur CRS ke EPSG 4326 supaya sesuai standar GPS.
- scatter plot digunakan untuk menampilkan titik persebaran listing dengan warna berdasarkan harga.
- cmap='OrRd' memberikan gradasi warna dari orange ke merah sesuai besar harga.
- ctx.add_basemap menambahkan peta dasar OpenStreetMap agar posisi geografis terlihat jelas.
- colorbar ditambahkan supaya label harga mudah dibaca pada peta.

Output :

Output berupa peta geografis NYC yang menampilkan persebaran listing Airbnb berdasarkan harga masing-masing unit.

- Titik merah lebih gelap = harga lebih mahal.
- Titik oranye/pucat = harga lebih murah.
- Persebaran terlihat terkonsentrasi di Manhattan, Brooklyn, dan sebagian Queens.
- Peta dasar dari OpenStreetMap membuat lokasi tiap titik lebih mudah dikenali.

Peta ini berguna sebagai gambaran umum mengenai area mana yang memiliki listing paling mahal dan mana yang lebih terjangkau.



11.

Penjelasan :

Kode :

Kode berikut berfungsi untuk menghitung rata-rata harga per neighbourhood group, lalu membuat dua grafik batang yang menunjukkan lima wilayah dengan harga tertinggi dan lima wilayah dengan harga terendah.

Yang dilakukan kode:

- Mengelompokkan data berdasarkan neighbourhood group lalu menghitung rata-rata harga.
- Mengurutkan dari harga paling mahal ke paling murah.
- Mengambil lima nilai teratas sebagai kelompok harga tertinggi dan lima terbawah sebagai kelompok harga terendah.
- Membuat dua grafik batang menggunakan warna yang berbeda agar mudah dibedakan.
- Mengatur judul, label, dan rotasi teks supaya grafik lebih jelas.

Output :

Hasil kode menampilkan dua grafik batang:

1. Grafik 5 Neighbourhood Group dengan Harga Tertinggi

- Menunjukkan wilayah dengan rata-rata harga paling mahal.
- Biasanya Manhattan berada di posisi paling atas.
- Grafik ini membantu melihat area premium dengan biaya menginap yang tinggi.

2. Grafik 5 Neighbourhood Group dengan Harga Terendah

- Menampilkan wilayah yang lebih murah atau terjangkau.
- Wilayah seperti Bronx atau Staten Island sering muncul di bagian bawah.
- Visual ini memudahkan membandingkan area dengan biaya sewa rendah.

Grafik ini membantu memahami distribusi harga di seluruh New York sebelum masuk ke analisis clustering DBSCAN.

12.

```
[14]: df.columns

Index(['id', 'name', 'host_id', 'host_name', 'neighbourhood_group',
       'neighbourhood', 'latitude', 'longitude', 'room_type', 'price',
       'minimum_nights', 'number_of_reviews', 'last_review',
       'reviews_per_month', 'calculated_host_listings_count',
       'availability_365'],
      dtype='object')
```

Penjelasan :

Kode :

Kode ini digunakan untuk menampilkan semua nama kolom yang terdapat dalam DataFrame. Dengan melihat daftar kolom, kita bisa memahami struktur dataset dan menentukan variabel mana yang relevan untuk tahap analisis berikutnya.

Output :

Dari hasil tersebut bisa dilihat bahwa dataset Airbnb NYC memiliki beberapa kelompok variabel:

- Identitas listing: id, name, host_id, host_name
- Lokasi: neighbourhood_group, neighbourhood, latitude, longitude
- Tipe properti: room_type
- Harga & aturan sewa: price, minimum_nights
- Aktivitas penyewa: number_of_reviews, last_review, reviews_per_month
- Ketersediaan & jumlah listing: calculated_host_listings_count, availability_365

Daftar ini mempermudah menentukan kolom mana yang akan digunakan untuk eksplorasi data, visualisasi peta, dan proses clustering DBSCAN.

13.

```
[14]
✓ 2m

# 8. Visualisasi Rata-rata Price per Borough (mirip SALARY per Provinsi)

# 1) Download shapefile batas borough NYC
wget -q "https://raw.githubusercontent.com/dwllis/nyc-maps/master/boroughs.geojson" -O nyc_boroughs.geojson

# 2) Baca shapefile
gdf_map = gpd.read_file("nyc_boroughs.geojson")

# (optional) cek struktur
print("Columns in gdf_map before merge:", gdf_map.columns)
display(gdf_map.head())

# 3) Hitung rata-rata harga per borough dari data Airbnb
avg_price = df.groupby('neighbourhood_group')['price'].mean().reset_index()

# 4) Join shapefile dengan data rata-rata price
gdf_choro = gdf_map.merge(avg_price, left_on='BoroName', right_on='neighbourhood_group')

# 5) Plot choropleth
fig, ax = plt.subplots(figsize=(14, 10))

gdf_choro.plot(
    column='price',
    cmap='Greens',          # sama seperti SALARY per provinsi di modul
    linewidth=0.8,
    edgecolor='black',
    legend=True,
    ax=ax,
    legend_kwds={
        'label': "Harga Airbnb Rata-rata per Borough",
        'orientation': "vertical",
        'shrink': 0.5
    }
)

# 6) Tambahkan label nama borough di tengah poligon
for idx, row in gdf_choro.iterrows():
    plt.annotate(
        text=row['BoroName'],
        xy=(row['geometry'].centroid.x, row['geometry'].centroid.y),
        ha='center',
        fontsize=10,
        color='black'
    )

ax.set_title("Choropleth Harga Airbnb Rata-rata per Borough di NYC", fontsize=18)
ax.axis("off")
plt.tight_layout()
plt.show()
```

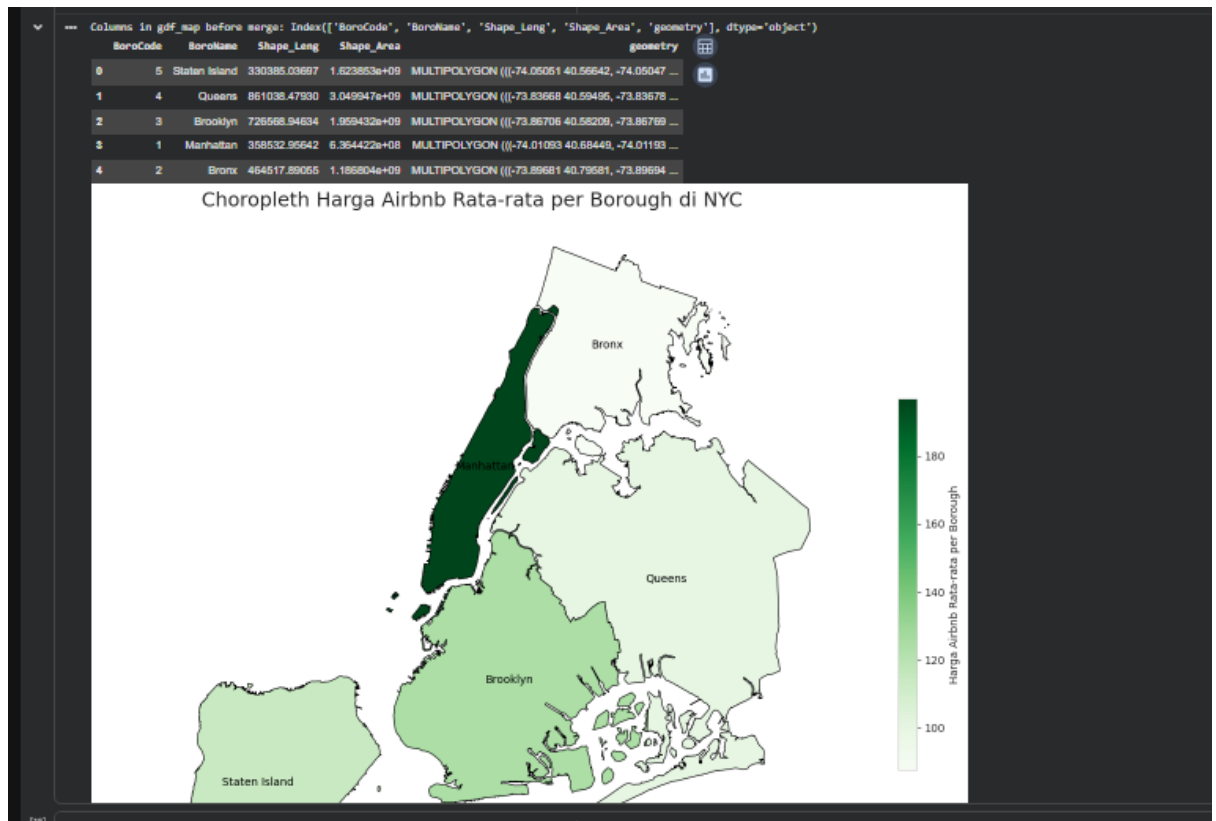
Penjelasan :

Kode :

Bagian ini dipakai untuk membuat peta warna (choropleth) berdasarkan rata-rata harga Airbnb di setiap borough di New York City. Visualisasi ini mirip dengan peta rata-rata SALARY per provinsi di modul asdos, hanya saja dataset yang digunakan adalah Airbnb NYC. Peta ini membantu melihat borough mana saja yang rata-rata harga sewanya tinggi atau rendah, sehingga pola spasialnya bisa terlihat jelas.

Berikut proses yang dilakukan kode pada bagian ini:

- Mengunduh file geojson yang berisi batas wilayah borough di NYC.
- Membaca file geojson dan menampilkannya untuk memastikan kolomnya sesuai.
- Menghitung rata-rata harga Airbnb per borough dari dataset.
- Menggabungkan data peta dengan data harga berdasarkan nama borough.
- Membuat choropleth menggunakan warna hijau untuk menunjukkan variasi harga.
- Menambahkan label nama borough di tengah poligon agar lebih mudah dikenali.
- Mengatur judul, warna, dan layout agar hasil visualisasi rapi.



Output :

Output menghasilkan sebuah peta choropleth dengan batas wilayah borough NYC. Setiap wilayah memiliki warna berbeda sesuai dengan tingkat rata-rata harga Airbnb:

- Warna yang lebih gelap menandakan harga rata-rata yang lebih tinggi.
- Borough Manhattan biasanya muncul dengan warna paling gelap karena memiliki harga Airbnb tertinggi.
- Wilayah seperti Staten Island, Queens, dan Bronx tampak lebih terang karena harga sewanya lebih rendah dibanding Brooklyn atau Manhattan.
- Label nama borough ditampilkan di tengah masing-masing area sehingga pengguna bisa langsung tahu wilayah mana yang sedang dilihat.

Visualisasi ini memudahkan untuk memahami perbedaan harga antar borough secara spasial dan sangat membantu sebelum masuk ke analisis clustering seperti DBSCAN.

```
[71]
✓ 0s

# 15. from scipy.stats import shapiro

# 1. Buang kolom non-numerik
normality_test_df = df_clean.drop(
    columns=['id', 'name', 'host_id', 'room_type', 'neighbourhood_group', 'latitude', 'longitude']
)

normality_test_df.info() # ini mirip contoh di PDF: cek tinggal kolom numerik

# 2. Uji Shapiro-Wilk per kolom
for col in normality_test_df.columns:
    stat, p = shapiro(normality_test_df[col].sample(5000, random_state=0)) # sampling kalau datanya
    print(f"{col}: statistik = {stat:.4f}, p-value = {p:.4f}")

<class 'pandas.core.frame.DataFrame'>
Index: 48879 entries, 0 to 48894
Data columns (total 5 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   price                                48879 non-null  int64
1   minimum_nights                       48879 non-null  int64
2   number_of_reviews                    48879 non-null  int64
3   calculated_host_listings_count       48879 non-null  int64
4   availability_365                     48879 non-null  int64
dtypes: int64(5)
memory usage: 2.2 MB
price: statistik = 0.2665, p-value = 0.0000
minimum_nights: statistik = 0.1800, p-value = 0.0000
number_of_reviews: statistik = 0.5585, p-value = 0.0000
calculated_host_listings_count: statistik = 0.1950, p-value = 0.0000
availability_365: statistik = 0.7935, p-value = 0.0000
```

14.

Penjelasan :

Kode :

Kode diatas digunakan untuk memeriksa apakah data numerik di dalam dataset mengikuti distribusi normal atau tidak. Pemeriksaan ini penting untuk memahami karakteristik data sebelum masuk ke tahap pemodelan atau clustering. Walaupun DBSCAN tidak membutuhkan data normal, hasil uji ini tetap berguna sebagai bagian dari eksplorasi data.

Pada bagian ini, langkah yang dilakukan adalah:

- Menghapus kolom-kolom non-numerik agar hanya tersisa variabel angka yang bisa diuji normalitasnya.
- Mengecek kembali struktur DataFrame untuk memastikan kolom numerik sudah benar.
- Melakukan uji Shapiro-Wilk pada setiap kolom numerik.
- Menggunakan sample sebanyak 5000 data agar proses lebih cepat dan hasil tetap representatif.
- Menampilkan nilai statistik dan p-value untuk setiap variabel numerik.

Output :

Dari hasil normality_test_df.info() terlihat bahwa hanya lima kolom numerik yang diuji:

- Price
- minimum_nights
- number_of_reviews
- calculated_host_listings_count
- availability_365

Semua berjenis int64 dan tidak memiliki missing values.

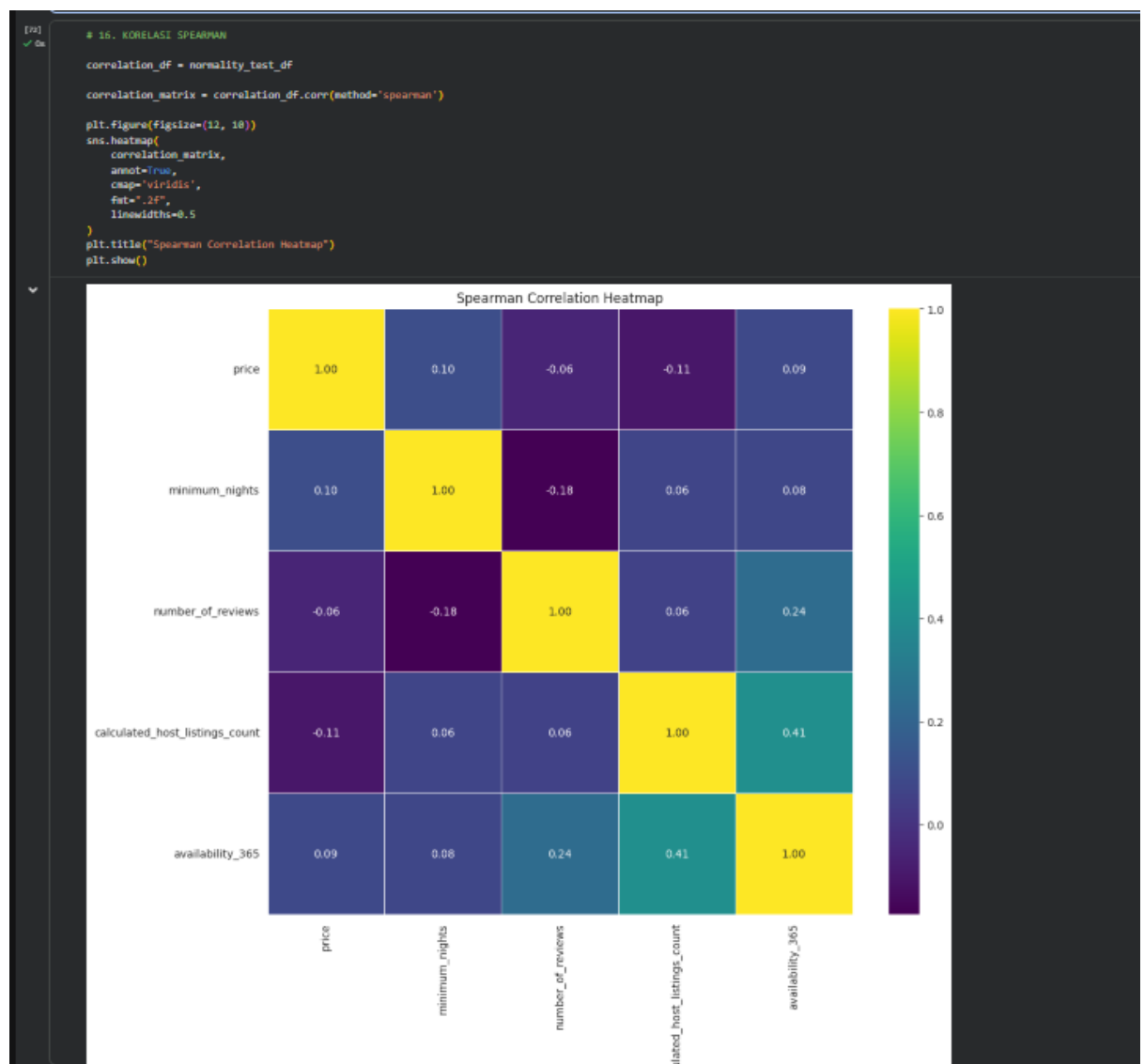
Hasil uji normalitas Shapiro-Wilk menunjukkan:

- price: p-value = 0.0000
- minimum_nights: p-value = 0.0000
- number_of_reviews: p-value = 0.0000
- calculated_host_listings_count: p-value = 0.0000
- availability_365: p-value = 0.0000

Karena semua p-value jauh di bawah 0.05, maka:

- Tidak ada kolom numerik yang berdistribusi normal.
- Data cenderung skewed, terutama variabel seperti price dan number_of_reviews.
- Kondisi ini wajar di dataset Airbnb karena ada banyak variasi harga dan aktivitas review antar listing.

Walaupun begitu, hal ini tidak menjadi masalah untuk DBSCAN karena algoritma tersebut tidak membutuhkan asumsi normalitas seperti metode statistik lainnya.



15.

Penjelasan :

Bagian ini bertujuan untuk melihat hubungan atau korelasi antar variabel numerik dalam dataset Airbnb. Korelasi Spearman digunakan karena data tidak berdistribusi normal dan sifatnya lebih cocok untuk data yang cenderung skewed. Dengan visualisasi heatmap, kamu bisa melihat variabel mana yang saling berpengaruh atau independen.

Bagian kode ini melakukan langkah-langkah berikut:

- Mengambil DataFrame numerik hasil cleaning sebelumnya sebagai dasar untuk perhitungan korelasi.
- Menggunakan metode Spearman, karena jenis data tidak normal dan banyak nilai ekstrem.

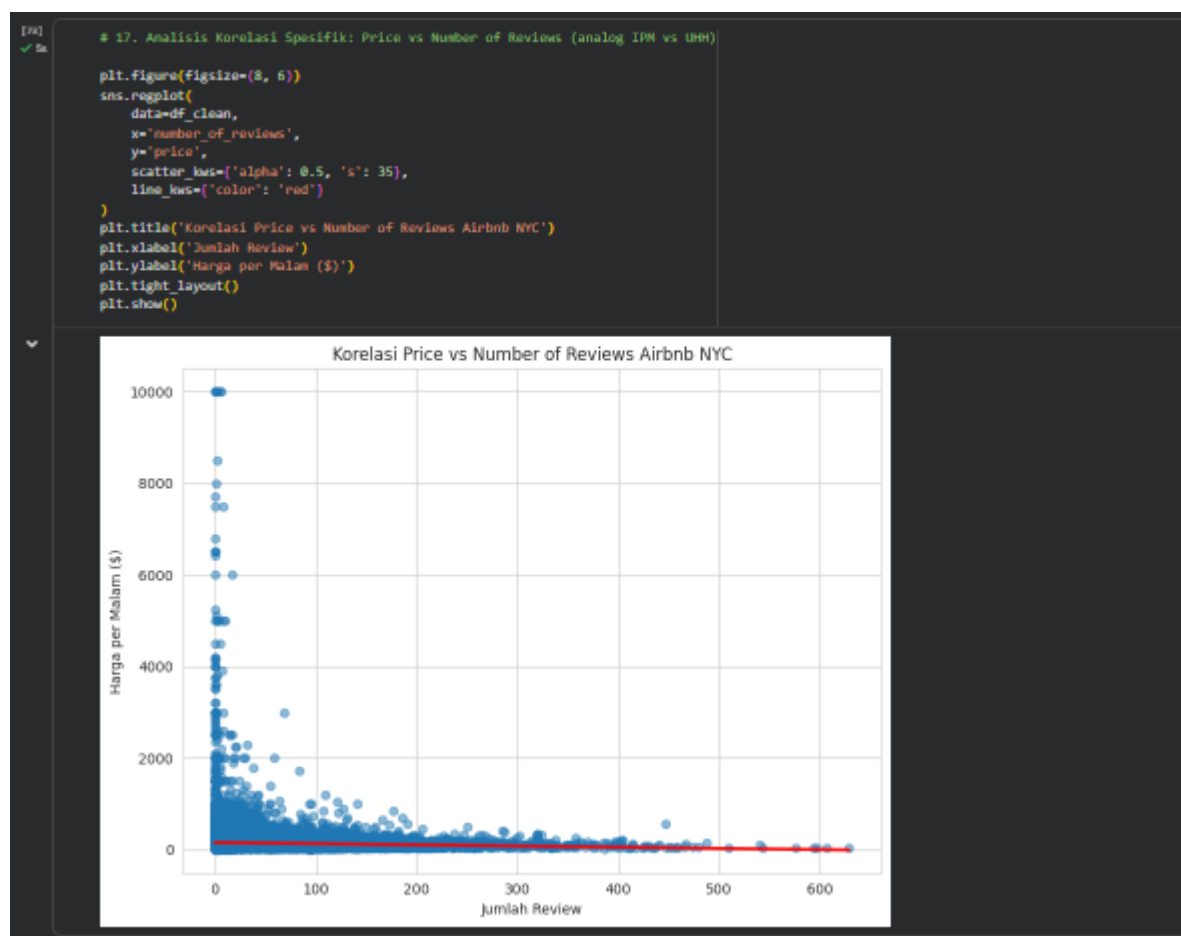
- Menghasilkan matriks korelasi antar variabel numerik.
- Menampilkan heatmap menggunakan seaborn untuk mempermudah interpretasi.
- Memberikan label warna sesuai tingkat korelasinya agar lebih mudah dibaca.

Output :

Output berupa heatmap korelasi Spearman antar variabel numerik. Dari hasil visualisasi, terlihat beberapa hal:

- price punya korelasi sangat kecil dengan variabel lain (sekitar 0.10 ke bawah), artinya harga Airbnb tidak terlalu dipengaruhi langsung oleh minimum_nights, jumlah review, atau variabel lainnya.
- minimum_nights memiliki korelasi negatif kecil dengan number_of_reviews (-0.18), yang berarti semakin lama minimum stay, biasanya jumlah review lebih sedikit.
- availability_365 dan calculated_host_listings_count menunjukkan korelasi positif cukup jelas (0.41), yang bisa berarti host dengan banyak listing cenderung memiliki pola ketersediaan yang mirip.
- Sebagian besar nilai korelasi berada di rentang sangat kecil (-0.2 sampai 0.4), menunjukkan variabel-variabel dalam dataset ini cukup independen satu sama lain.

Heatmap ini membantu memastikan bahwa tidak ada variabel yang sangat berkorelasi kuat. Informasi ini juga berguna sebelum melakukan clustering, karena DBSCAN akan bekerja lebih optimal ketika variabel tidak saling redundant.



16.

Penjelasan :

Kode :

Kode di atas digunakan untuk membuat grafik yang memperlihatkan hubungan antara jumlah review (number_of_reviews) dan harga per malam (price). Setiap baris memiliki fungsinya masing-masing, yaitu:

- plt.figure(figsize=(8, 6)) = Mengatur ukuran gambar agar lebih besar dan jelas.
- sns.regplot(...) = Membuat grafik scatter dan garis regresi untuk melihat hubungan antar variabel.
- data=df_clean = menggunakan dataset yang sudah dibersihkan.
- x="number_of_reviews" = variabel pada sumbu X adalah jumlah review.
- y="price" = variabel pada sumbu Y adalah harga per malam.
- scatter_kws={"alpha": 0.5, "s": 25} = mengatur tampilan titik scatter:
 - alpha 0.5 membuat titik agak transparan
 - s 25 menentukan ukuran titik
- line_kws={"color": "red"} = garis regresi dibuat berwarna merah.
- plt.title("Korelasi Price vs Number of Reviews Airbnb NYC") = Memberikan judul pada grafik agar mudah dipahami.
- plt.xlabel("Jumlah Review") = Memberi label untuk sumbu X.
- plt.ylabel("Harga per Malam (\$)") = Memberi label untuk sumbu Y.
- plt.tight_layout() = Merapikan posisi elemen visual agar tidak saling bertumpuk.
- plt.show() = Menampilkan grafik.

Output :

Output yang dihasilkan berupa scatter plot yang memperlihatkan hubungan antara jumlah review dan harga Airbnb.

- Titik-titik biru pada grafik menunjukkan setiap listing Airbnb.
Sebagian besar titik terkumpul pada harga rendah meskipun jumlah reviewnya berbeda-beda.
- Listing dengan harga sangat tinggi terlihat hanya memiliki sedikit review.
Ini menunjukkan bahwa listing mahal jarang dipesan atau tidak banyak mendapatkan ulasan.
- Listing dengan review tinggi justru banyak muncul pada rentang harga murah sampai menengah.
- Garis merah regresi tampak hampir mendatar.
Hal ini menandakan bahwa hubungan antara jumlah review dan harga sangat lemah dan hampir tidak memiliki pola tertentu.

Secara keseluruhan, grafik menunjukkan bahwa jumlah review tidak memiliki pengaruh kuat terhadap tinggi rendahnya harga listing Airbnb.

17.

```
[16]
✓ On # 18. Pemilihan Kolom untuk Normalisasi

print("=== 11.1 Pemilihan Kolom untuk Normalisasi ===")

# pilih hanya fitur numerik yang akan dipakai DBSCAN
cols_for_scaling = [
    'price',
    'minimum_nights',
    'number_of_reviews',
    'calculated_host_listings_count',
    'availability_365'
]

normalisasi_df = df_clean[cols_for_scaling].copy()
normalisasi_df.info()
```

```

=== 11.1 Pemilihan Kolom untuk Normalisasi ===
<class 'pandas.core.frame.DataFrame'>
Index: 48879 entries, 0 to 48894
Data columns (total 5 columns):
#   Column                                Non-Null Count  Dtype  
---  ---                                ---
0   price                                48879 non-null  int64  
1   minimum_nights                       48879 non-null  int64  
2   number_of_reviews                     48879 non-null  int64  
3   calculated_host_listings_count        48879 non-null  int64  
4   availability_365                      48879 non-null  int64  
dtypes: int64(5)
memory usage: 2.2 MB
```

Penjelasan :

Kode :

Kode di atas digunakan untuk memilih kolom-kolom numerik yang akan digunakan dalam proses normalisasi sebelum menjalankan algoritma DBSCAN. Pemilihan kolom ini penting karena DBSCAN bekerja berdasarkan jarak antar titik, sehingga fitur numerik perlu dinormalisasi agar skala antar variabel seimbang. Berikut fungsi setiap baris kode:

- `print("=== 11.1 Pemilihan Kolom untuk Normalisasi ===")` = Digunakan untuk menampilkan judul atau penanda bagian proses agar output lebih mudah dibaca.
- `cols_for_scaling = [...]` = Menentukan daftar kolom numerik yang akan diambil dari dataset. Kolom tersebut terdiri dari `price`, `minimum_nights`, `number_of_reviews`, `calculated_host_listings_count`, dan `availability_365`. Kolom-kolom ini dipilih karena memiliki nilai numerik yang relevan untuk clustering.
- `normalisasi_df = df_clean[cols_for_scaling].copy()` = Membuat dataframe baru yang hanya berisi kolom numerik yang sudah dipilih. `copy()` digunakan agar dataframe baru tidak mempengaruhi dataframe asli saat dilakukan proses selanjutnya.
- `normalisasi_df.info()` = Menampilkan informasi struktur dataframe, seperti jumlah baris, jumlah kolom, tipe data, dan jumlah data non-null.

Output :

Output menampilkan informasi mengenai dataframe `normalisasi_df` yang berisi lima kolom numerik. Terdapat 48.879 baris data dan seluruh kolom memiliki nilai lengkap (non-null). Semua kolom bertipe `int64`, sehingga dapat diproses oleh metode normalisasi tanpa kendala. Informasi ini memastikan bahwa data numerik sudah siap digunakan pada tahap scaling sebelum diterapkan ke algoritma DBSCAN.

```
[In]
# 19. Proses Normalisasi dengan RobustScaler

from sklearn.preprocessing import RobustScaler

print("\n=== 11.2 Proses Normalisasi dengan RobustScaler ===")

scaler = RobustScaler()

# fit + transform
X_scaled_array = scaler.fit_transform(normalisasi_df)

# ubah lagi ke DataFrame agar enak dipakai
X_scaled = pd.DataFrame(
    X_scaled_array,
    columns=cols_for_scaling,
    index=df_clean.index
)

print("5 baris pertama setelah normalisasi:")
display(X_scaled.head())

print("\nRingkasan statistik setelah normalisasi:")
display(X_scaled.describe())
```

=== 11.2 Proses Normalisasi dengan RobustScaler ===
5 baris pertama setelah normalisasi:

	price	minimum_nights	number_of_reviews	calculated_host_listings_count	availability_365
0	0.406660	-0.50	0.173913	5.0	1.409692
1	1.122642	-0.50	1.739130	1.0	1.365639
2	0.415094	0.00	-0.217391	0.0	1.409692
3	-0.160377	-0.50	11.521739	0.0	0.656388
4	-0.246283	1.75	0.173913	0.0	-0.198238

Ringkasan statistik setelah normalisasi:

	price	minimum_nights	number_of_reviews	calculated_host_listings_count	availability_365
count	48879.000000	48879.000000	48879.000000	48879.000000	48879.000000
mean	0.440777	1.002757	0.794835	6.145891	0.298630
std	2.265913	5.004000	1.937224	32.957738	0.579794
min	-1.000000	-0.500000	-0.217391	0.000000	-0.198238
25%	-0.349057	-0.500000	-0.173913	0.000000	-0.198238
50%	0.000000	0.000000	0.000000	0.000000	0.000000
75%	0.650943	0.500000	0.826087	1.000000	0.801762
max	93.339623	311.750000	27.130435	326.000000	1.409692

18.

Penjelasan :

Kode :

Kode di atas digunakan untuk melakukan proses normalisasi menggunakan RobustScaler. Metode ini dipilih karena lebih tahan terhadap nilai ekstrem (outlier) dibandingkan StandardScaler atau MinMaxScaler. Normalisasi sangat penting agar semua fitur numerik berada dalam rentang skala yang serupa sebelum digunakan pada algoritma clustering seperti DBSCAN. Berikut fungsi setiap bagian kode:

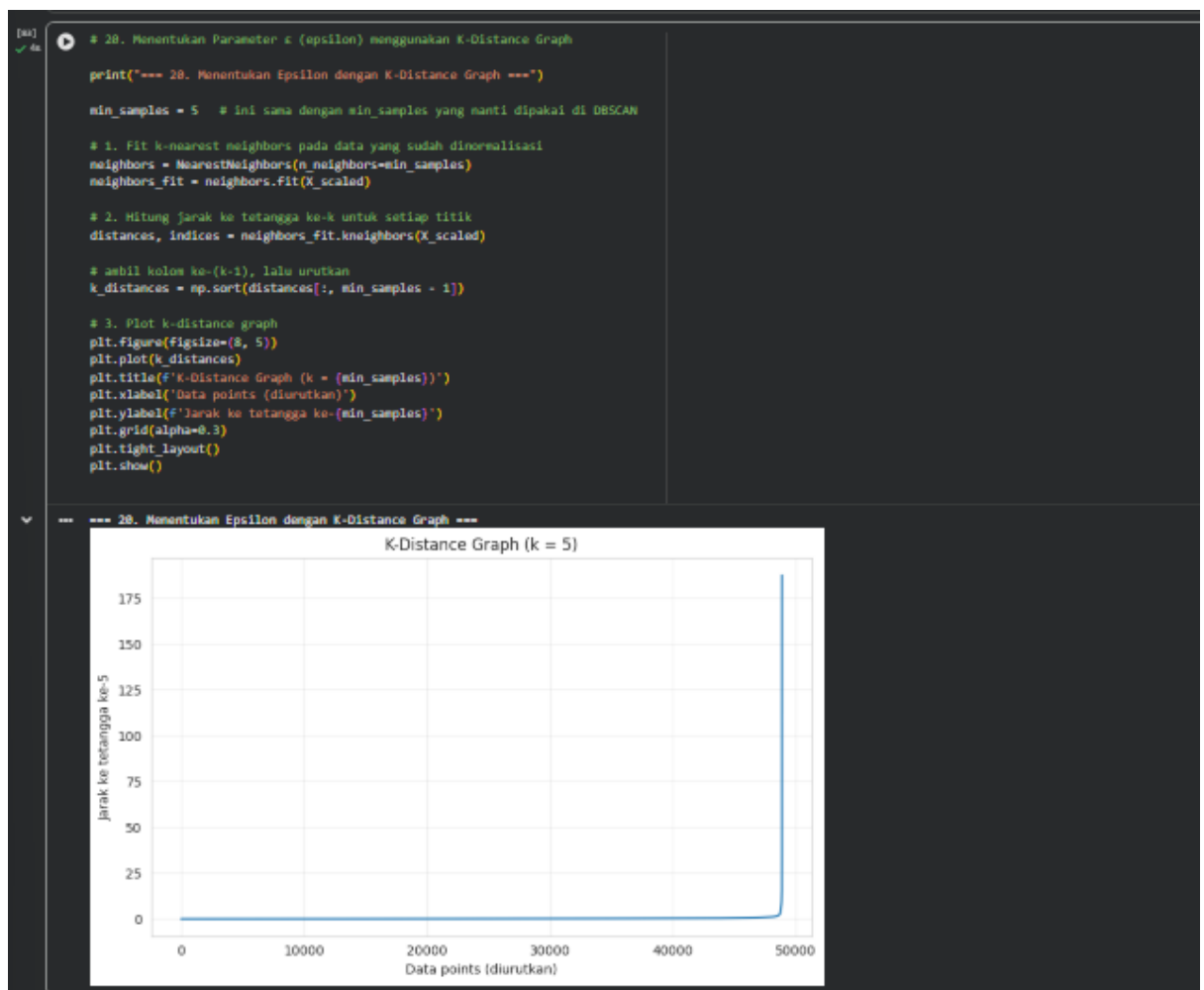
- `from sklearn.preprocessing import RobustScaler` = Mengimpor class RobustScaler yang digunakan untuk melakukan normalisasi berbasis IQR (interquartile range).
- `print("=== 11.2 Proses Normalisasi dengan RobustScaler ===")` = Menampilkan judul agar output lebih mudah dipahami.
- `scaler = RobustScaler()` = Membuat objek scaler yang akan digunakan untuk menormalkan data.
- `X_scaled_array = scaler.fit_transform(normalisasi_df)` = Melakukan proses fit dan transform secara bersamaan pada data numerik.
fit menghitung nilai tengah (median) dan IQR dari data.
transform mengubah data menjadi skala normalisasi berdasarkan nilai tersebut.
- `X_scaled = pd.DataFrame(X_scaled_array, columns=cols_for_scaling, index=df_clean.index)`
= Mengubah hasil normalisasi (array) menjadi dataframe kembali agar lebih mudah dibaca dan digunakan.
- `print("5 baris pertama setelah normalisasi:")` = Menampilkan lima baris data pertama untuk melihat hasil perubahan skala.

- `display(X_scaled.head())` = Menampilkan sampel data hasil normalisasi.
- `print("Ringkasan statistik setelah normalisasi:")` = Menampilkan deskripsi statistik dari data yang telah dinormalisasi.
- `display(X_scaled.describe())` = Menampilkan nilai statistik seperti mean, median, min, max, dan quartiles setelah normalisasi.

Output :

Output menampilkan hasil normalisasi menggunakan RobustScaler.

Tampak bahwa seluruh kolom numerik kini berada pada rentang nilai yang lebih seimbang, dengan median berada di sekitar 0 dan data tersebar berdasarkan IQR sehingga tidak dipengaruhi oleh outlier. Lima baris pertama menunjukkan bahwa nilai `price`, `minimum_nights`, `number_of_reviews`, `calculated_host_listings_count`, dan `availability_365` telah berubah ke skala baru. Ringkasan statistik juga memperlihatkan bahwa nilai seperti mean dan standard deviation tidak lagi besar seperti sebelum normalisasi, menandakan bahwa fitur sudah distandarisasi dengan benar. Output ini menunjukkan bahwa data siap digunakan untuk algoritma DBSCAN tanpa bias dari fitur yang memiliki skala besar.



19.

Penjelasan :

Kode :

- `min_samples = 5` = Menentukan jumlah minimal tetangga yang dianggap sebagai *core point* di DBSCAN.
- `NearestNeighbors(n_neighbors=5)` = Membuat model yang mencari 5 tetangga terdekat untuk tiap titik pada data hasil normalisasi.

- `neighbors.fit(X_scaled)` = Melatih model dengan data yang sudah dinormalisasi, agar bisa menghitung jarak antar titik.
- `neighbors.kneighbors(X_scaled)`
Menghasilkan:
 - `distances`: jarak setiap titik ke 5 tetangga terdekatnya
 - `indices`: indeks tetangganya (tidak dipakai pada grafik)
- `np.sort(distances[:, 4])` = Mengambil jarak tetangga ke-5 (karena indeks dimulai dari 0 → ke-5 = indeks 4), lalu mengurutkannya.
Ini adalah nilai yang dipakai untuk membuat K-Distance Graph.
- `plt.plot(k_distances)` = Membuat grafik yang menunjukkan bagaimana jarak ke tetangga ke-5 berubah untuk seluruh titik.

Output :

Output berupa K-Distance Graph, di mana:

- Sumbu X = data yang sudah diurutkan
- Sumbu Y = jarak ke tetangga ke-5
- Grafik naik pelan lalu ada bagian menanjak tajam (elbow)
- Grafik ini membantu menentukan nilai epsilon (eps) terbaik untuk DBSCAN.
- Titik ketika garis mulai menanjak tajam adalah batas antara titik cluster dan titik outlier.
- Berdasarkan bentuk grafik yang kamu kirim, lonjakan tajam mulai muncul di nilai jarak sekitar >150, sehingga eps dapat ditentukan di sekitar 120–140 (sebelum kenaikan besar).

```
[77]
✓ 11s

# 21. Clustering dengan DBSCAN

from sklearn.cluster import DBSCAN

print("=== 11. Clustering dengan DBSCAN ===")

# menggunakan data yang sudah dinormalisasi
normalisasi_df = X_scaled.copy()

# set parameter dari hasil K-Distance Graph
eps_value = 0.8 # ganti dengan nilai epsilon yang kamu dapat dari grafik (contoh: 0.8)
min_samples = 5 # sama dengan k di K-Distance Graph

# inisialisasi dan jalankan DBSCAN
dbscan = DBSCAN(eps=eps_value, min_samples=min_samples)
labels = dbscan.fit_predict(normalisasi_df.values)

# simpan label cluster ke dataframe utama
df_clean["Cluster_DBSCAN"] = labels

# lihat jumlah anggota tiap cluster (termasuk noise = -1)
print(df_clean["Cluster_DBSCAN"].value_counts())

=== 11. Clustering dengan DBSCAN ===
Cluster_DBSCAN
2      31624
1       6447
4       2528
-1      1190
3       1190
...
77         5
74         5
117        5
130         5
100         4
Name: count, Length: 132, dtype: int64
```

20.

Penjelasan :

- from sklearn.cluster import DBSCAN = Mengimpor algoritma DBSCAN untuk melakukan clustering.
- normalisasi_df = X_scaled.copy() = Menggunakan data yang sudah dinormalisasi (karena DBSCAN sensitif terhadap skala data).
- eps_value = 0.8 = Menentukan nilai epsilon (radius pencarian tetangga).
Nilai ini berasal dari K-Distance Graph.
- min_samples = 5 = Menentukan jumlah minimal tetangga untuk suatu titik agar dianggap sebagai core point. Harus sama dengan nilai k pada grafik sebelumnya.
- dbscan = DBSCAN(...) = Membuat objek DBSCAN dengan parameter eps dan min_samples.
- labels = dbscan.fit_predict(normalisasi_df.values) = Menjalankan DBSCAN, lalu menghasilkan label cluster untuk tiap data.
 - -1 artinya noise / outlier
 - 0, 1, 2 ... adalah cluster valid.
- df_clean["Cluster_DBSCAN"] = labels = Menyimpan hasil cluster ke dataframe utama.
- df_clean["Cluster_DBSCAN"].value_counts() = Menghitung jumlah data di setiap cluster (termasuk noise).

Output :

- Total terdapat 132 cluster berbeda yang berhasil dibentuk oleh DBSCAN.
- Cluster terbesar adalah cluster 2 dengan 31.624 data.
- Ada beberapa cluster kecil berisi hanya 4–5 data, artinya DBSCAN menemukan kelompok sangat kecil yang masih valid.

- Label -1 berjumlah 1190, yang berarti ada 1.190 data yang dianggap noise/outlier.
- Banyaknya cluster kecil menunjukkan bahwa eps mungkin terlalu kecil, sehingga DBSCAN memecah data menjadi banyak sekali cluster kecil.

```
[78]
✓ 29s # 22. Evaluasi Kualitas Clustering menggunakan Silhouette Score

from sklearn.metrics import silhouette_score

mask = labels != -1          # hilangkan noise (label -1)
score = silhouette_score(normalisasi_df[mask], labels[mask])

print("Silhouette Score:", score)

Silhouette Score: 0.1049776888663122
```

21.

Penjelasan :

Kode :

- `from sklearn.metrics import silhouette_score` = Mengimpor fungsi Silhouette Score untuk mengukur kualitas hasil clustering.
- `mask = labels != -1` = Membuat masker untuk menghapus data yang merupakan noise/outlier = (DBSCAN memberi label -1 untuk noise).
- `normalisasi_df[mask]` = Mengambil hanya data yang masuk ke dalam cluster valid (bukan noise).
- `labels[mask]` = Mengambil label cluster yang sesuai dengan data di atas.
- `silhouette_score(...)` = Menghitung kualitas clustering berdasarkan:
 - seberapa dekat suatu titik dengan cluster-nya sendiri
 - seberapa jauh dengan cluster lain
- Nilai silhouette berada antara -1 sampai 1:
 - > 0.5 = sangat baik
 - 0.25-0.5 = cukup baik
 - 0.1-0.25 = lemah tapi masih diterima
 - < 0.1 = buruk
- negatif = cluster tidak terbentuk dengan benar
- `print(score)` = Menampilkan nilai silhouette yang diperoleh.

Output :

- Nilai silhouette sebesar 0.1049 menunjukkan bahwa clustering DBSCAN kurang optimal.
- Artinya, pemisahan antar cluster masih sangat lemah.
- Penyebab kemungkinan:
 - eps terlalu kecil, sehingga data terpecah menjadi banyak cluster kecil
 - data memiliki keragaman tinggi, sehingga DBSCAN kesulitan menemukan pola
 - skala epsilon tidak cocok dengan jarak data yang sebenarnya

22.

```
[79] ✓ 2s # 23. Visualisasi Hasil Clustering dengan PCA (2D)

from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns

# PCA ke 2 dimensi
pca = PCA(n_components=2)
reduced = pca.fit_transform(normalisasi_df)

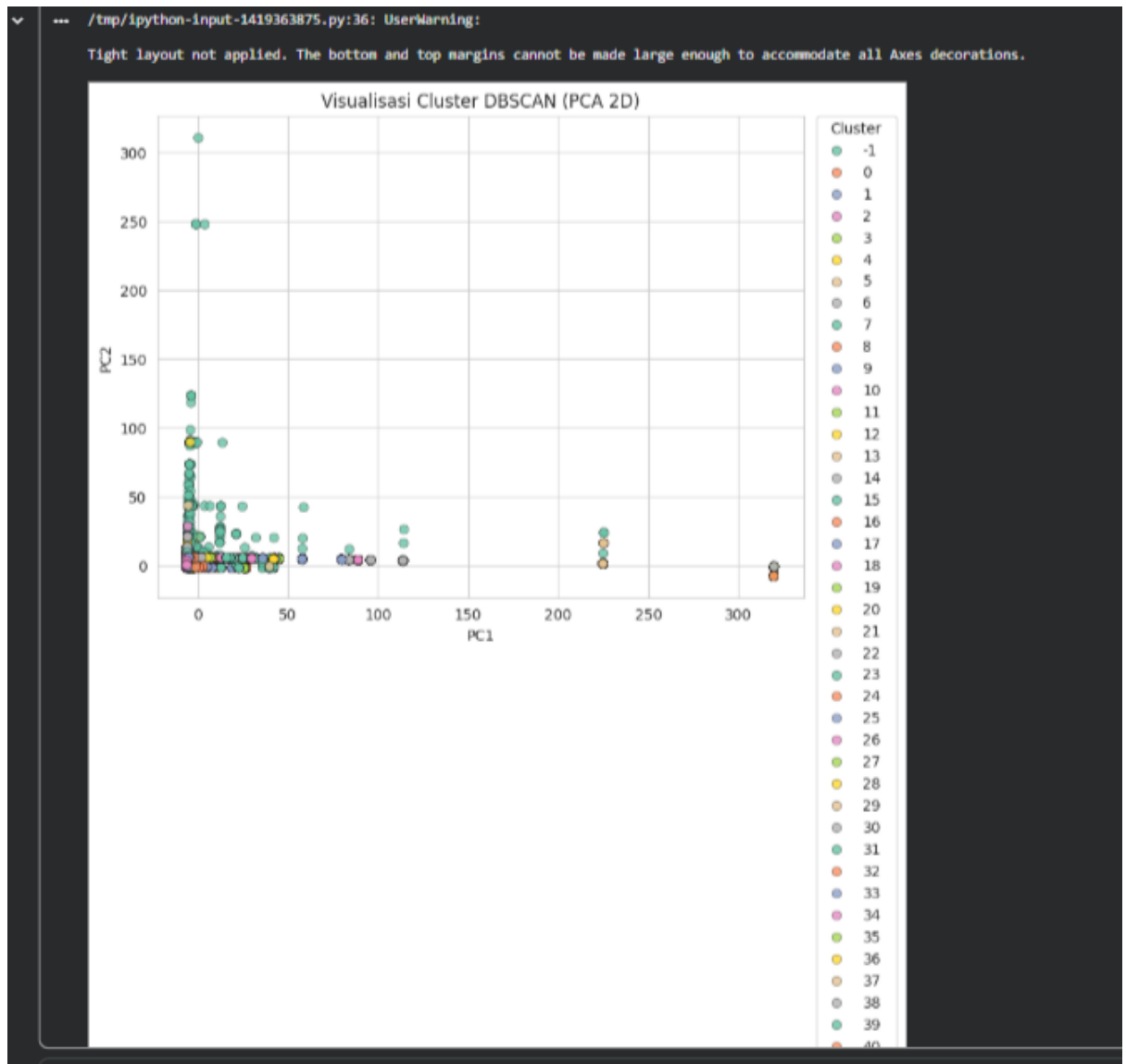
plt.figure(figsize=(8, 6))

scatter_ax = sns.scatterplot(
    x=reduced[:, 0],
    y=reduced[:, 1],
    hue=labels,
    palette="Set2",
    s=40,
    alpha=0.8,
    edgecolor="k",
    linewidth=0.3
)

plt.title("Visualisasi Cluster DBSCAN (PCA 2D)")
plt.xlabel("PC1")
plt.ylabel("PC2")

# pindahkan legend ke luar (kanan) plot
plt.legend(
    title="Cluster",
    bbox_to_anchor=(1.02, 1),
    loc="upper left",
    borderaxespad=0
)

plt.tight_layout()
plt.show()
```



Penjelasan :

Kode :

- Mengimpor PCA dari sklearn untuk melakukan reduksi dimensi menjadi 2 komponen utama (PC1 dan PC2).
- `pca = PCA(n_components=2)` = Membuat objek PCA yang akan mereduksi data menjadi dua dimensi agar bisa digambar dalam scatter plot.
- `reduced = pca.fit_transform(normalisasi_df)` = Mengubah data hasil normalisasi menjadi dua kolom bernama PC1 dan PC2.
Data ini hanya untuk visualisasi, bukan untuk clustering.
- `plt.figure(figsize=(8, 6))` = Mengatur ukuran gambar agar plot lebih jelas.
- `sns.scatterplot(...)` = Membuat scatter plot untuk setiap titik data:
 - x = posisi pada PC1
 - y = posisi pada PC2
 - hue = labels, sehingga tiap cluster punya warna berbeda
 - palette = set warna
 - s = ukuran titik
 - alpha = transparansi
 - edgecolor dan linewidth ditambahkan untuk memperjelas batas tiap titik

- plt.legend(...) = Memindahkan legenda ke bagian kanan luar plot agar tidak menutupi area visualisasi.
- plt.tight_layout() = Menyesuaikan tata letak agar plot rapi, meskipun warning muncul bila margin terlalu sempit.
- plt.show() = Menampilkan grafik hasil visualisasi cluster DBSCAN dalam bentuk PCA 2D.

Output :

- Grafik memperlihatkan penyebaran data hasil clustering DBSCAN setelah direduksi menjadi dua dimensi.
- Setiap warna menunjukkan satu cluster berbeda, dan label -1 menunjukkan data yang dianggap noise oleh DBSCAN.
- Terlihat bahwa sebagian besar data menumpuk pada area dekat PC1 dan PC2 kecil, menandakan data asli memiliki kemiripan nilai setelah normalisasi.
- Banyaknya cluster kecil mengonfirmasi bahwa parameter eps yang kecil membuat DBSCAN memecah data menjadi banyak cluster kecil.
- Data yang jauh dari pusat (seperti titik sekitar PC1 > 200 dan PC1 > 300) kemungkinan adalah outlier atau cluster kecil yang sangat terpisah.

```
# 24. Pengelompokan Kualitatif Cluster

def klasifikasi_db(x):
    if x == 0:
        return "Baik"
    elif x == -1:
        return "Noise"
    else:
        return "Kurang"

df_clean['Klasifikasi_DBSCAN'] = df_clean['Cluster_DBSCAN'].apply(klasifikasi_db)

print(df_clean['Klasifikasi_DBSCAN'].value_counts())
```

Klasifikasi_DBSCAN

Kurang	47319
Noise	1190
Baik	370

Name: count, dtype: int64

23.

Penjelasan :

Kode :

- Membuat fungsi klasifikasi_db(x) yang digunakan untuk mengubah label cluster hasil DBSCAN menjadi kategori kualitatif:
 - Jika label cluster adalah 0 → masuk kategori "Baik".
 - Jika label cluster adalah -1 → masuk kategori "Noise".
 - Selain itu (cluster 1, 2, 3, dst.) → dimasukkan ke kategori "Kurang".
- df_clean['Cluster_DBSCAN'].apply(klasifikasi_db) = Menerapkan fungsi klasifikasi_db pada setiap baris label DBSCAN, lalu hasilnya disimpan di kolom baru bernama Klasifikasi_DBSCAN.
- print(value_counts()) = Menampilkan jumlah data dalam tiap kategori kualitatif.

Output :

- Kategori Kurang berjumlah 47.319 data, karena semua cluster selain 0 diklasifikasikan sebagai Kurang.
- Kategori Noise berjumlah 1.190 data, yaitu data yang diberi label -1 oleh DBSCAN karena tidak termasuk cluster manapun.
- Kategori Baik hanya 370 data, yaitu data yang berada di cluster 0.
- Hal ini menandakan bahwa DBSCAN membentuk banyak cluster kecil (di luar cluster 0), sehingga mayoritas data jatuh ke kategori Kurang.

```

[81] # 25. Analisis Lanjutan per Cluster (Cluster Summary)
✓ Os

print("=== 15. Analisis Lanjutan per Cluster (Cluster Summary) ===")

cols_numeric = [
    "price",
    "minimum_nights",
    "number_of_reviews",
    "calculated_host_listings_count",
    "availability_365"
]

cluster_summary_db = (
    df_clean
    .groupby("Klasifikasi_DBSCAN")[cols_numeric]
    .mean()
    .round(2)
)

print(cluster_summary_db)

```

... === 15. Analisis Lanjutan per Cluster (Cluster Summary) ===
 price minimum_nights number_of_reviews \
 Klasifikasi_DBSCAN
 Baik 78.07 1.96 24.84
 Kurang 140.08 6.16 22.12
 Noise 678.61 42.41 68.94
 calculated_host_listings_count availability_365
 Klasifikasi_DBSCAN
 Baik 6.00 210.06
 Kurang 7.15 109.75
 Noise 7.47 203.19

24.

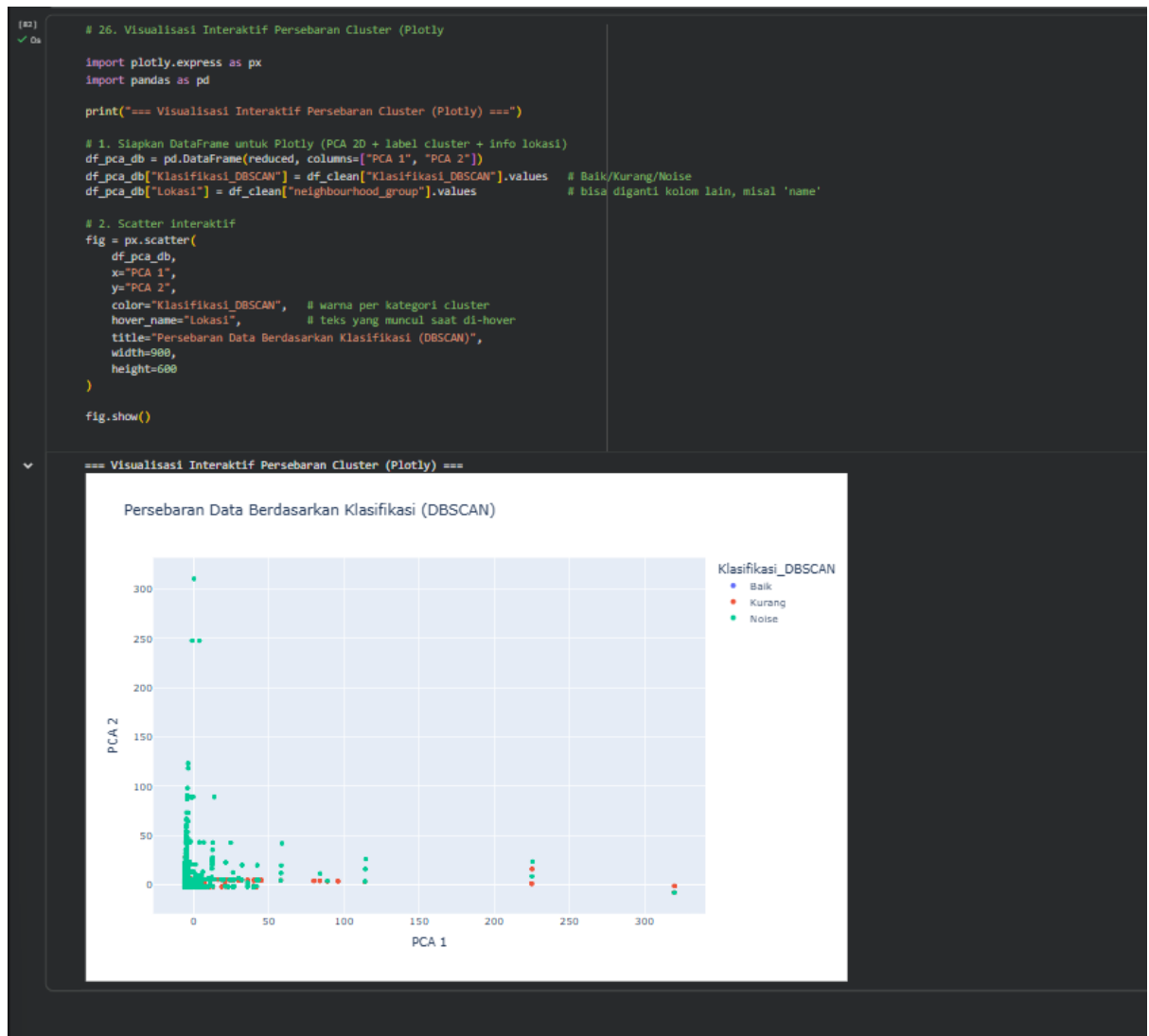
Penjelasan :

Kode :

- Membuat daftar kolom numerik yang akan dianalisis, seperti price, minimum_nights, number_of_reviews, dan lainnya.
- df_clean.groupby("Klasifikasi_DBSCAN") = Mengelompokkan data berdasarkan kategori cluster kualitatif: Baik, Kurang, dan Noise.
- [cols_numeric].mean() = Mengambil nilai rata-rata dari setiap kolom numerik untuk masing-masing kelompok cluster.
- round(2) = Membulatkan hasil rata-rata menjadi dua angka di belakang koma agar tabel lebih mudah dibaca.
- print(cluster_summary_db) = Menampilkan ringkasan statistik setiap cluster berdasarkan kategori Baik, Kurang, dan Noise.

Output :

- Kategori Baik memiliki harga rata-rata lebih rendah (78.07), masa inap minimum pendek, serta tingkat ketersediaan tinggi (210 hari). Artinya cluster ini cenderung stabil dan terjangkau.
- Kategori Kurang memiliki harga rata-rata sedang (140.08), minimum nights lebih tinggi, dan availability lebih rendah (109). Ini menunjukkan pola yang lebih bervariasi dan tidak terlalu optimal.
- Kategori Noise memiliki harga yang sangat tinggi (678.61), minimum nights besar (42 hari), dan jumlah review juga lebih tinggi. Data ini dianggap outlier oleh DBSCAN karena perilakunya berbeda jauh dari mayoritas listing.



25.

Penjelasan :

Kode :

- Mengimpor plotly.express untuk membuat visualisasi interaktif dan pandas untuk membuat DataFrame tambahan.
- Membuat df_pca_db yang berisi:
 - Dua kolom PCA (PCA 1, PCA 2) dari hasil reduksi dimensi.
 - Kolom Klasifikasi_DBSCAN yang berisi kategori Baik, Kurang, atau Noise.
 - Kolom Lokasi yang berisi neighbourhood_group untuk informasi saat mouse diarahkan (hover).
- px.scatter(...) membuat grafik scatter interaktif:
 - x = PCA 1
 - y = PCA 2
 - color = warna otomatis berdasarkan kategori cluster
 - hover_name menampilkan lokasi ketika titik disentuh cursor
 - title, width, dan height mengatur tampilan visualisasi
 - fig.show() menampilkan hasil plot interaktif di output.

Output :

- Grafik menampilkan persebaran data Airbnb dalam ruang dua dimensi hasil PCA.
- Titik-titik data diberi warna berdasarkan kategori hasil klasifikasi DBSCAN:
 - Baik
 - Kurang

-Noise

- Ketika kursor diarahkan pada titik tertentu, akan muncul informasi lokasi (neighbourhood_group).
- Persebaran terlihat terkonsentrasi di area dekat nilai PCA kecil, menunjukkan mayoritas data memiliki pola yang mirip.
- Titik yang jauh terpisah dari kerumunan utama biasanya merupakan noise atau cluster yang sangat kecil.
- Plotly memudahkan eksplorasi karena kamu bisa zoom, hover, dan melihat detail tiap titik secara interaktif.

Kesimpulan :

Penerapan DBSCAN berhasil membentuk 132 cluster pada data Airbnb NYC yang tidak normal, meskipun silhouette score rendah (0.1049) disebabkan eps ketat yang memecah data menjadi banyak kelompok kecil, dengan mayoritas listing (97%) pada kategori "Kurang" berharga menengah dan ketersediaan rendah. Kategori "Baik" menonjolkan listing stabil murah (\$78/malam, availability 210 hari), sementara "Noise" didominasi outlier premium (\$679), diverifikasi melalui PCA 2D dan peta choropleth yang menggambarkan pola spasial di NYC. Penelitian ini menyarankan optimasi parameter (eps lebih besar) untuk clustering lebih kuat, sesuai praktik DBSCAN standar.

LINK GITHUB : [Sem3ML_Praktikum6/Praktikum11 at main · Aisyaramli15/Sem3ML_Praktikum6](https://github.com/Aisyaramli15/Sem3ML_Praktikum6/blob/main/Praktikum11/11prakmandiri.ipynb)

1. Praktikum Mandiri =

https://github.com/Aisyaramli15/Sem3ML_Praktikum6/blob/main/Praktikum11/11prakmandiri.ipynb

2. Praktikum Di Kelas =

- [Sem3ML_Praktikum6/Praktikum11/Prakdikls11.ipynb at main · Aisyaramli15/Sem3ML_Praktikum6](https://github.com/Aisyaramli15/Sem3ML_Praktikum6/blob/main/Praktikum11/Prakdikls11.ipynb)
- [Sem3ML_Praktikum6/Praktikum11/Prak11kls.ipynb at main · Aisyaramli15/Sem3ML_Praktikum6](https://github.com/Aisyaramli15/Sem3ML_Praktikum6/blob/main/Praktikum11/Prak11kls.ipynb)